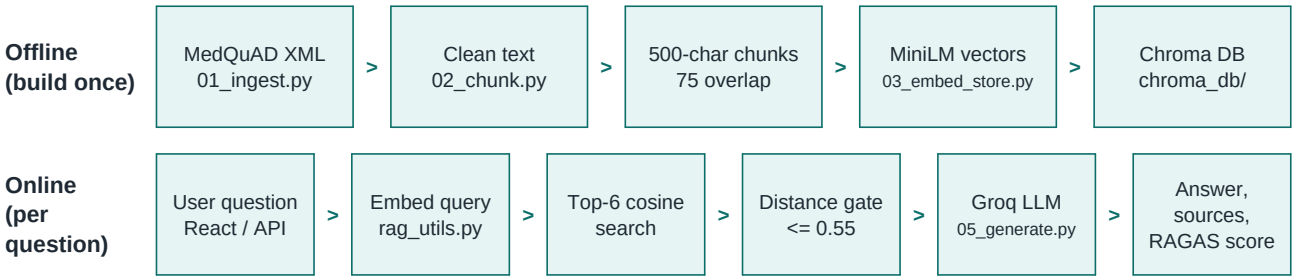
# How the Health FAQ RAG Pipeline Works

A stage-by-stage walkthrough of what the code does, with file locations, settings and the measured results. Every stage on the requested list is covered, including the ones this project does not implement yet. Each section ends with the code (from the repo, or a labelled example where a stage is not built) and a line-by-line explanation in plain words.

### The system in one paragraph

The chatbot answers consumer cancer-health questions using only NIH content. Offline, it parses the **MedQuAD CancerGov** XML files (729 question-answer pairs across 98 cancer topics), cleans them, splits the answers into **6,452 chunks**, embeds each chunk with **all-MiniLM-L6-v2** and stores the vectors in a **Chroma** database. At question time it embeds the question, fetches the 6 nearest chunks, drops any that are too far away, and either refuses (nothing close enough) or sends the chunks to **openai/gpt-oss-120b on Groq** with a strict "answer only from the context" prompt. Each answer is scored live with **RAGAS** (faithfulness, relevance, hallucination) and saved to SQLite.




### Technology stack

| Layer | What is used |
|---|---|
| Data | MedQuAD, CancerGov subset (NIH), parsed with Python's xml.etree |
| Chunking | LangChain RecursiveCharacterTextSplitter |
| Embeddings | sentence-transformers/all-MiniLM-L6-v2 (384-dim, runs locally) |
| Vector store | ChromaDB, persistent on disk, HNSW index, cosine distance |
| Generator LLM | openai/gpt-oss-120b via the Groq API (Cloudflare Falcon-7B as a switchable backup) |
| Evaluation | ROUGE-1, BERTScore, RAGAS (judge: openai/gpt-oss-20b on Groq) |
| App | FastAPI backend, SQLite chat history, React (Vite) UI, Streamlit demo |

## 1. Data Ingestion

**What it means:** getting raw source documents into a uniform, machine-readable form.

**IN THIS PROJECT** src/01_ingest.py writes data/qa_pairs.json

- Reads every XML file in `MedQuAD/1_CancerGov_QA/`. Each file is one cancer topic containing several `<QAPair>` elements.
- For every pair it keeps: `doc_id, qid, qtype` (e.g. symptoms, treatment, exams and tests), `focus` (the topic, e.g. "Leukemia"), `source, url, question, answer`.
- Robustness: files that fail to parse are skipped with a message; pairs with an empty question or answer are dropped.
- Result: **729 Q-A pairs over 98 topics**. Using a trusted, curated source (NIH) is the first safety decision in a health chatbot: the model can only be as reliable as its corpus.

### The code, explained (simplified from src/01_ingest.py)

```
tree = ET.parse(filepath)
root = tree.getroot()
focus = root.find("Focus").text.strip()

for qa_pair in root.findall(".//QAPair"):
    question = (qa_pair.find("Question").text or "").strip()
    answer = (qa_pair.find("Answer").text or "").strip()
    if not question or not answer:
        continue
    records.append({"doc_id": doc_id, "qid": qid, "qtype": qtype,
                    "focus": focus, "source": source, "url": url,
                    "question": question, "answer": answer})
```

| Code | In plain words |
|---|---|
| `ET.parse(filepath)` | Opens one XML file and turns it into a tree of tags that Python can walk through. |
| `root.find("Focus")` | Finds the topic name, e.g. "Leukemia". Every Q-A pair in this file gets that label. |
| `root.findall(".//QAPair")` | Finds every question-answer block anywhere in the file. The **for** loop handles them one at a time. |
| `(... .text or "").strip()` | Takes the text inside a tag. **or ""** avoids a crash when the tag is empty; **strip()** trims spaces at the ends. |
| `if not question or not answer: continue` | Skips broken pairs instead of saving empty records. |
| `records.append({...})` | Saves the pair as a dictionary (named fields). **main()** later writes the whole list to data/qa_pairs.json. |

## 2. Data Preprocessing

**What it means:** cleaning and enriching text so that chunks and embeddings are good quality.

**IN THIS PROJECT** clean_text() in src/02_chunk.py and enrich_for_embedding() in src/rag_utils.py

- **Whitespace normalisation.** MedQuAD answers contain messy line breaks and indentation. `re.sub(r"\s+", " ", text)` collapses all of it into single spaces.
- **Metadata carried forward.** Every chunk keeps its topic, source URL, original question and qtype, so answers can show where they came from.
- **Contextual enrichment before embedding.** A chunk is not embedded on its own. It is embedded as `"Topic: {focus}. Question: {question}. Answer: {chunk}"`. A chunk from the middle of an answer may never name the disease; adding the topic and question makes its vector land near the questions users actually ask. Only the plain chunk text is stored and shown to the LLM.
- Missing `qtype` values become an empty string, because Chroma metadata cannot hold `None`.

Side effect worth knowing: because all newlines are removed *before* splitting, the splitter's paragraph and line separators never match, so chunks are cut on sentence boundaries (". ") and then words.

### The code, explained (src/02_chunk.py and src/rag_utils.py)

```
def clean_text(text):
    text = re.sub(r"\s+", " ", text)
    return text.strip()

def enrich_for_embedding(focus, question, chunk_text):
    return f"Topic: {focus}. Question: {question}. Answer: {chunk_text}"
```

| Code | In plain words |
|---|---|
| `re.sub(r"\s+", " ", text)` | **\s+** means "one or more spaces, tabs or newlines". Each such run is replaced by a single space. |
| `text.strip()` | Removes leftover spaces at the start and end. |

| Code | In plain words |
|---|---|
| `f"Topic: {focus}. ..."` | An f-string fills each {} slot with a real value, e.g. "Topic: Leukemia. Question: What are the symptoms of Leukemia?. Answer: ...". |
| `(used only for the vector)` | This combined text is what gets embedded. The text stored in the database and shown to the LLM stays the plain chunk. |

## 3. Chunking

**What it means:** splitting long documents into small passages so that retrieval returns focused evidence and the prompt stays short.

**IN THIS PROJECT** src/02_chunk.py writes data/chunks.json

- **Recursive splitting** tries the largest natural boundary first and only falls back to smaller ones when a piece is still over 500 characters, so sentences are rarely cut in half.
- **75-character overlap** means a fact that straddles a boundary still appears whole in at least one chunk.
- Each answer is chunked separately (a chunk never mixes two topics). IDs look like `rec12_0000001_1_c0`, with a numeric suffix added if an ID would repeat. Chunks also record `chunk_index` and `total_chunks`.
- Result: 729 answers become **6,452 chunks** (about 8.9 per answer).

### The code, explained (simplified from src/02_chunk.py)

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=75,
    separators=["\n\n", "\n", ". ", " ", ""],
)
for idx, record in enumerate(qa_pairs):
    pieces = splitter.split_text(clean_text(record["answer"]))
    for i, piece in enumerate(pieces):
        chunk_id = f"rec{idx}_{record['qid']}_c{i}"
        all_chunks.append({"chunk_id": chunk_id, "focus": record["focus"],
                           "question": record["question"], "chunk_text": piece,
                           "chunk_index": i, "total_chunks": len(pieces)})
```

| Code | In plain words |
|---|---|
| `chunk_size=500, chunk_overlap=75` | Each piece is at most 500 characters; neighbouring pieces share 75 characters. |
| `separators=[...]` | Preferred cut points, best first: paragraph break, line break, end of sentence, space, and as a last resort anywhere. |
| `enumerate(qa_pairs)` | Loops over the answers and also counts them (**idx**), which is used in the chunk ID. |
| `splitter.split_text(...)` | Returns a list of strings: the chunks of this one answer. |
| `f"rec{idx}_{record['qid']}_c{i}"` | Builds a unique name, e.g. rec12_0000001_1_c0 = record 12, chunk 0. |
| `all_chunks.append({...})` | Stores the chunk with its labels. The full list is written to data/chunks.json. |

## 4. Embeddings and Vector Database

**What it means:** turning each chunk into a vector whose direction captures meaning, then storing the vectors in an index that can find nearest neighbours quickly.

**IN THIS PROJECT** src/03_embed_store.py and src/rag_utils.py

### Embeddings

- Model: **sentence-transformers/all-MiniLM-L6-v2**, a small (about 22M parameter) model that outputs 384-dimensional vectors and runs on a laptop CPU. No API key or per-call cost.
- Vectors are **L2-normalised** (`normalize_embeddings=True`), so cosine similarity is a plain dot product.

- Encoded in batches of 256. The model is loaded once and reused (see section 9).
- Questions use the same model, so queries and chunks live in the same vector space. Note the asymmetry: chunks are embedded with the Topic/Question prefix, the user's question is embedded as typed.

### Vector database: Chroma

- `chromadb.PersistentClient` stores everything on disk in `chroma_db/` (gitignored; rebuilt by `run.py setup`).
- Collection `medquad_cancergov` uses an **HNSW** approximate nearest-neighbour index with `hnsw:space = cosine`. Each record holds the ID, the vector, the chunk text and the metadata.
- The build script deletes and recreates the collection, so re-running it never leaves duplicate or stale chunks.

### The code, explained (src/rag_utils.py and src/03_embed_store.py)

```
_embedder = None

def get_embedder():
    global _embedder
    if _embedder is None:
        _embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
    return _embedder

def encode_texts(texts):
    vectors = get_embedder().encode(texts, normalize_embeddings=True)
    return vectors.tolist()

collection = client.create_collection(
    name="medquad_cancergov",
    metadata={"hnsw:space": "cosine"},
)
collection.add(ids=ids, documents=documents,
               metadatas=metadatas, embeddings=vectors)
```

| Code | In plain words |
|---|---|
| `if _embedder is None:` | Loads the model only the first time it is needed and reuses it afterwards (a simple cache). |
| `SentenceTransformer(...)` | Downloads (first run only) and loads the MiniLM model, which turns any text into 384 numbers. |
| `.encode(texts, normalize_embeddings=True)` | Converts a list of texts into vectors, scaled to length 1 so cosine similarity is easy to compute. |
| `.tolist()` | Converts the NumPy array into plain Python lists, which Chroma accepts. |
| `create_collection(... "cosine")` | Creates a new collection (like a table) that measures closeness with cosine distance. |
| `collection.add(...)` | Saves each chunk's ID, text, labels and vector. The real script does this 256 chunks at a time. |

## 5. Query Expansion

**What it means:** rewriting or enlarging the user's question before searching (synonyms, several paraphrases, or a hypothetical answer) so that more relevant chunks are found.

> **NOT BUILT** The question is only whitespace-trimmed and embedded as-is.

What partly compensates today: the document-side enrichment from section 2. Because every chunk vector already contains the canonical MedQuAD question ("What are the symptoms of Leukemia?"), short or casual user questions still land close to the right chunks.

### How it could be added

- **Multi-query:** ask the LLM for 3 paraphrases, retrieve for each, merge the results.
- **HyDE:** have the LLM draft a hypothetical answer and embed that instead of the question.

- **Medical synonym map:** expand lay terms and abbreviations ("blood cancer" to leukemia, "NHL" to non-Hodgkin lymphoma) without an extra LLM call.

### The code, explained

```
# What the project does today (backend/rag_service.py)
retrieved = retrieve(question.strip(), collection)

# Example only, NOT in the project: multi-query expansion
variants = [question] + llm_paraphrases(question, n=3)
results = [retrieve(v, collection) for v in variants]
```

| Code | In plain words |
|---|---|
| `question.strip()` | The only change made to the question today: spaces at the ends are removed. |
| `llm_paraphrases(...)` | A made-up helper for the example: it would ask the LLM for 3 rewordings of the question. |
| `[retrieve(v, ...) for v in variants]` | Searches once per wording. Merging the results finds chunks that match any of the wordings. |

## 6. Retrieval

**What it means:** finding the chunks most likely to contain the answer.

**IN THIS PROJECT** retrieve() in src/rag_utils.py

- **Dense semantic search:** the 6 nearest chunks by cosine distance.
- **Relevance threshold:** any chunk farther than 0.55 is discarded. If none survive, the question is treated as **out of the knowledge base** and the LLM is never called (`is_likely_out_of_kb()`). This one number is the main out-of-scope detector.
- **Context formatting:** surviving chunks are joined with separators, each labelled `[Source: {topic} | CancerGov]` so the LLM can cite them.
- The API returns every retrieved chunk with its URL and distance, and the UI shows them as sources under the answer.

### The code, explained (simplified from src/rag_utils.py)

```
def retrieve(question, collection, top_k=6, max_distance=0.55):
    query_embedding = encode_query(question)
    results = collection.query(query_embeddings=[query_embedding],
                               n_results=top_k)
    retrieved = []
    for doc, meta, dist in zip(results["documents"][0],
                               results["metadatas"][0],
                               results["distances"][0]):
        if dist <= max_distance:
            retrieved.append((doc, meta, dist))
    return retrieved

def format_context(retrieved):
    blocks = [f"[Source: {m['focus']} | {m['source']}]\n{d}"
              for d, m, _ in retrieved]
    return "\n\n---\n\n".join(blocks)
```

| Code | In plain words |
|---|---|
| `encode_query(question)` | Turns the user's question into a vector with the same model used for the chunks. |
| `collection.query(..., n_results=top_k)` | Asks Chroma for the 6 closest chunks. It returns lists of texts, labels and distances. |
| `[0]` | Chroma can search for many questions at once; **[0]** picks the results for our single question. |

| Code | In plain words |
|---|---|
| `zip(...)` | Walks through the three lists side by side, so each text stays with its labels and distance. |
| `if dist <= max_distance:` | The relevance filter: a chunk is kept only if it is close enough (distance 0.55 or less). |
| `return retrieved` | This list may be empty. Empty means "out of scope", and the bot refuses. |
| `format_context(...)` | Joins the chunks into one block of text, each with a [Source: ...] label, separated by ---. |

## 7. Hybrid Search and Re-Ranking

**What it means:** *hybrid search* combines keyword search (BM25 / TF-IDF, good at exact drug names and rare terms) with vector search (good at meaning). *Re-ranking* re-scores the candidates with a stronger model, usually a cross-encoder that reads the question and chunk together.

> **NOT BUILT** Retrieval is vector-only, and chunks are used in Chroma's distance order.

Housekeeping note: `data/tfidf_vectorizer.pkl` is in the repository and `scikit-learn` is in requirements.txt, but no current code loads either. They look like leftovers from an earlier keyword-search experiment.

### How it could be added

- **Hybrid:** run BM25 (e.g. the `rank_bm25` package) over `chunks.json` next to the Chroma query, then merge the two ranked lists with **Reciprocal Rank Fusion**: score = sum of 1 / (60 + rank).
- **Re-rank:** fetch around 20 candidates, re-score them with `cross-encoder/ms-marco-MiniLM-L-6-v2`, keep the best 6.
- The evaluation already measures context precision and recall (section 10), so any change here can be checked before and after.

### The code, explained (example only, NOT in the project)

```
# Reciprocal Rank Fusion: merge a vector ranking and a keyword ranking
def rrf(vector_ids, bm25_ids, k=60):
    scores = {}
    for ranked in (vector_ids, bm25_ids):
        for rank, chunk_id in enumerate(ranked, start=1):
            scores[chunk_id] = scores.get(chunk_id, 0) + 1 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)
```

| Code | In plain words |
|---|---|
| `vector_ids, bm25_ids` | Two ranked lists of chunk IDs: one from Chroma (meaning), one from keyword search (exact words). |
| `1 / (k + rank)` | A chunk earns more points the higher it ranks. k=60 stops either list from dominating. |
| `scores.get(chunk_id, 0) + ...` | A chunk found by both searches collects points from both, so it rises to the top. |
| `sorted(..., reverse=True)` | The merged order, best first. A cross-encoder could then re-score the top 20 and keep 6. |

## 8. Generation

**What it means:** the LLM writes the final answer, conditioned on the retrieved context.

> **IN THIS PROJECT** src/05_generate.py, called from backend/rag_service.py

| Setting | Value | Why |
|---|---|---|
| Model | openai/gpt-oss-120b on Groq | Strong open model; Groq gives a free tier and low latency |
| temperature | 0 | Deterministic, no creative wording in medical answers |
| max_tokens | 500 | Keeps answers concise; bounds cost and latency |
| Messages | system prompt + user message | User message = "Context: ... Question: ..." |
| Backup | Cloudflare Workers AI, Falcon-7B | Switch by changing GENERATION_BACKEND |

- If the context is empty, `generate_answer()` returns the fixed refusal text instead of calling the model.
- A missing `GROQ_API_KEY` raises a clear error, which the API turns into HTTP 503.
- The API response contains the answer, a `refused` flag, the sources and the RAGAS metrics. `frontend/src/formatAnswer.jsx` renders the answer text.

### The code, explained (simplified from src/05_generate.py)

```
def answer_question(question, collection):
    retrieved = retrieve(question, collection, top_k=6)
    context = format_context(retrieved)
    if is_likely_out_of_kb(retrieved):
        return OUT_OF_KB_REPLY
    return generate_answer(question, context)

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"},
    ],
    temperature=0,
    max_tokens=500,
)
return response.choices[0].message.content
```

| Code | In plain words |
|---|---|
| `answer_question(...)` | The whole RAG flow in five lines: search, format, then refuse or answer. |
| `is_likely_out_of_kb(retrieved)` | True when no chunk passed the distance filter. The fixed refusal is returned and the LLM is never called. |
| `os.environ.get("GROQ_API_KEY")` | Reads the secret key from the .env file, so the key never appears in the code. |
| `"role": "system"` | The rules the model must follow (see section 11). |
| `"role": "user"` | The retrieved chunks and the question, sent together in one message. |
| `temperature=0, max_tokens=500` | No randomness, and the answer is capped at 500 tokens (roughly 375 words). |
| `response.choices[0].message.content` | Pulls the answer text out of the API's reply. |

## 9. Memory Management and Caching

**What it means:** *memory* is what the bot remembers between turns (so a follow-up like "how is it treated?" works). *Caching* is reusing expensive work: loaded models, embeddings or whole answers.

> **PARTLY** Chat history is stored and models are cached in-process. No conversational memory and no answer cache.

### Chat history (stored, not fed back to the model)

- `backend/history.py` saves every exchange to SQLite (`data/chat_history.db`): timestamp, question, answer, refused flag, sources JSON, metrics JSON.
- Endpoints: `GET /api/history` (latest 50, max 200), `GET /api/history/{id}`, `DELETE /api/history`. The React sidebar lists past questions and can clear them.

- **Every question is answered independently.** Earlier turns are never sent to the LLM, so a follow-up that says "it" is searched without knowing what "it" was.

### Caching

- The embedding model is loaded once per process (global `_embedder` in rag_utils.py), because loading it takes seconds.
- The Chroma collection and the generation module are loaded once (`get_pipeline()` in rag_service.py).
- The RAGAS judge LLM and its embeddings are created once (`_get_judge_and_embeddings()`).
- Chunk embeddings are computed once at build time and persisted in Chroma; only the question is embedded at runtime.

Possible additions: rewrite follow-ups into standalone questions using the last few turns from SQLite before retrieval; add an exact-match or semantic answer cache (for example, reuse an answer when a new question's embedding is within 0.05 of a cached one) to save Groq calls.

### The code, explained (backend/rag_service.py and backend/history.py)

```
_gen = None
_collection = None

def get_pipeline():
    global _gen, _collection
    if _gen is None:
        _gen = _load_generate()
        _collection = load_collection()
    return _gen, _collection

conn.execute(
    "INSERT INTO chat_history (created_at, question, answer, refused,"
    " sources_json, metrics_json) VALUES (?, ?, ?, ?, ?, ?)",
    (now, question, answer, 1 if refused else 0,
     json.dumps(sources), json.dumps(metrics)),
)
```

| Code | In plain words |
|---|---|
| `if _gen is None:` | The same "load once, then reuse" cache as the embedder. The first question is slow; later ones are fast. |
| `INSERT INTO chat_history ...` | Adds one row to the SQLite table for every question answered. |
| `? placeholders` | The values are passed separately from the SQL, which blocks SQL injection through text the user typed. |
| `json.dumps(sources)` | Lists and dictionaries are stored as JSON text, because the column holds plain text. |
| `1 if refused else 0` | SQLite has no true/false type, so the flag is stored as 1 or 0. |

## 10. Evaluation

**What it means:** measuring whether retrieval finds the right evidence, whether answers are correct and grounded, and whether the bot refuses what it should not answer.

**IN THIS PROJECT** eval/build_test_set.py, eval/metrics.py, src/run_eval.py, backend/ragas_metrics.py, eval/ragas_context_eval.py

### Test sets (Walert methodology)

| Category | How it is built | Size | Metric |
|---|---|---|---|
| Known | Up to 4 real MedQuAD questions per qtype, random seed 42 | 45 | ROUGE-1 and BERTScore F1 vs. the gold answer |
| Inferred | Hand-written questions that need two chunk types (e.g. symptoms + diagnosis) | 3 | Answered (not refused) rate |
| Out-of-KB | Off-topic questions (capital of France, bicycle tyres...) | 5 | Correct refusal rate |

### RAGAS metrics (LLM-as-judge)

- **Faithfulness:** share of the answer's claims supported by the retrieved chunks.
- **Answer relevancy:** how well the answer addresses the question.
- **Hallucination** = 1 - faithfulness (lower is better).
- **Context precision / recall:** are the retrieved chunks relevant, and do they cover the gold answer?
- Judge is **openai/gpt-oss-20b**, deliberately a different model from the generator so answers are not graded by their own author. Inline citation tags are stripped before judging.
- Live chat scores every answer; if RAGAS fails (e.g. rate limit) it falls back to a word-overlap heuristic and records why. Batch evaluation uses a "patient" mode with retries for the Groq free tier.

### Latest saved results (data/eval_results.json, data/ragas_context_results.json)

| Metric | Result | Metric | Result |
|---|---|---|---|
| Known questions answered | 43 / 45 (95.6%) | RAGAS faithfulness | 0.915 |
| Avg ROUGE-1 | 0.386 | RAGAS answer relevancy | 0.804 |
| Avg BERTScore F1 | 0.842 | Hallucination (1 - faith.) | 0.085 |
| Inferred answered | 3 / 3 | Context precision (n=15) | 0.805 |
| Out-of-KB refused | 5 / 5 (100%) | Context recall (n=15) | 0.717 |
| Out-of-scope sensitivity | 1.00 (5/5) | Specificity | 0.956 (43/45) |

RAGAS faithfulness and relevancy were scored for 17 of the 43 answered known questions (the rest hit free-tier limits and used the heuristic). The README quotes an older quick-eval run (80% answered, BERTScore 0.849); the figures above are from the saved result files. ROUGE-1 is low because answers are paraphrased summaries; BERTScore, which compares meaning, is the better signal here.

### The code, explained (eval/metrics.py, eval/build_test_set.py, backend/ragas_metrics.py)

```
def looks_like_refusal(text):
    lower = text.lower()
    return any(p in lower for p in REFUSAL_PHRASES)

rng = random.Random(42)
sample = rng.sample(records, KNOWN_PER_QTYPE)    # 4 per question type

result = evaluate(ds, metrics=[faithfulness, answer_relevancy],
                  llm=judge, embeddings=embeddings)
hallucination = 1.0 - faithfulness
```

| Code | In plain words |
|---|---|
| `any(p in lower for p in REFUSAL_PHRASES)` | True if the answer contains any known refusal phrase, such as "don't have enough information". |
| `random.Random(42)` | A fixed seed: the same test questions are picked on every run, so results can be compared. |
| `rng.sample(records, 4)` | Picks 4 random questions of each type (symptoms, treatment, ...). |
| `evaluate(...)` | RAGAS sends the question, answer and chunks to the judge LLM, which checks every claim against the chunks. |
| `1.0 - faithfulness` | If 92% of the claims are supported, the other 8% count as possible hallucination. |

## 11. Guardrails and Prompting

**What it means:** the controls that keep a health chatbot safe: staying inside its knowledge, refusing gracefully, citing sources, and rejecting bad input.

> **IN THIS PROJECT** Layered guardrails in retrieval, prompt, generation settings, API and evaluation

### The system prompt

```
You are a careful health-information assistant. Answer ONLY using the provided
context. If the context does not contain the answer, say clearly that you don't
have enough information to answer, and suggest the user consult a medical
professional. Do not speculate or use outside knowledge. Cite the source
focus/topic for each claim where possible.
```

Each line has a job: **role** (careful health assistant), **grounding** (only the context), **refusal path** (say so and point to a professional), **anti-hallucination** (no speculation or outside knowledge) and **attribution** (cite topics).

### Guardrail layers

| Layer | Mechanism | Where |
|---|---|---|
| Scope gate | No chunk within distance 0.55: fixed refusal, LLM not called | rag_utils.py, 05_generate.py |
| Grounding prompt | Answer only from context, refuse otherwise, cite sources | 05_generate.py |
| Decoding | temperature 0, max 500 tokens | 05_generate.py |
| Input validation | Question must be 1-500 characters (Pydantic) | backend/api.py |
| Access | CORS limited to localhost front-end origins; API key kept in .env | backend/api.py |
| Refusal detection | Phrase list identifies refusals for metrics and evaluation | eval/metrics.py |
| Output check | Live faithfulness / hallucination score shown with each answer | backend/ragas_metrics.py |

### Gaps to be aware of

- No prompt-injection filter: text like "ignore your instructions" reaches the model, which relies on the system prompt alone.
- No emergency detection: a message describing urgent symptoms gets a normal FAQ answer rather than a "seek urgent care" response.
- No standing medical disclaimer on every answer (only on refusals), and no PII handling for personal details users type into the stored history.
- Faithfulness is measured after the answer is shown, not used to block a low-scoring answer.

### The code, explained (backend/api.py and src/05_generate.py)

```
class ChatRequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=500)
    save_history: bool = True
    compute_metrics: bool = True

OUT_OF_KB_REPLY = ("I don't have enough information in my health knowledge "
                   "base to answer that question. Please consult a medical "
                   "professional or a trusted health resource.")
```

| Code | In plain words |
|---|---|
| `class ChatRequest(BaseModel)` | Pydantic describes what a valid request looks like. FastAPI checks every request against it automatically. |
| `Field(..., min_length=1, max_length=500)` | **...** means the question is required. Empty or over-500-character questions are rejected before any work is done. |
| `OUT_OF_KB_REPLY` | A fixed, pre-written refusal. Because the LLM does not generate it, it cannot hallucinate. |

## Summary

| Stage | Status | Key detail |
|---|---|---|
| Data ingestion | Built | MedQuAD CancerGov XML, 729 Q-A pairs, 98 topics |
| Preprocessing | Built | Whitespace cleanup; Topic + Question prefix for embedding |
| Chunking | Built | Recursive splitter, 500 chars, 75 overlap, 6,452 chunks |
| Embeddings | Built | all-MiniLM-L6-v2, 384-dim, normalised, local |
| Vector database | Built | Chroma, persistent, HNSW, cosine |
| Query expansion | Not built | Question used as typed |
| Retrieval | Built | Top 6, cosine distance threshold 0.55 |
| Hybrid search / re-ranking | Not built | Vector-only; unused TF-IDF pickle in data/ |
| Generation | Built | gpt-oss-120b on Groq, temperature 0, 500 tokens |
| Memory and caching | Partly | SQLite history, models cached; no follow-up memory or answer cache |
| Evaluation | Built | Known / inferred / out-of-KB; ROUGE, BERTScore, RAGAS |
| Guardrails and prompting | Built | Scope gate, grounding prompt, input limits; injection and emergency handling missing |